the respect to the variance of the two sets. The method assumes the input is already sorted.

#2/#3 Initialize temporary variables for the minimum variance and the index where to split the list.

#4 Goes through all the possible values for the split index: one constraint is that both sets need to have at least `m` points, so we can exclude all choices for which the first set has less than `m` elements, as well as those where the second set is too small.

#5/#6 For each possible value `i` for the split index, select the points before and after the split, and compute the variances of the two sets.

#7-#9 If the sum of the variances just computed is better than the best result so far, update the temporary variables.

#10 Return the best option found.

And with this, we can finally close section about `SsTree::insert`; you might feel this was a very long road to get here, and you'd be right: this is probably the most complicated code we described so far, take your time to read the last few sections even multiple times, if it helps, and then… brace yourself, we are going to delve into the `delete` method next, which is likely even more complicated.

10.3.5 Delete

As it was for `insert`, also `delete` in Ss-trees is heavily based on B-trees's `delete`; the former is normally considered so complicated that many textbooks skip it altogether (for the sake of space), and implementing it is usually avoided as